# DSAPS Assignment 1

**Deadline: 6 September 2026, 11:59pm**

## Important Points

1. The assignment has 3 questions. You need to do ALL THREE questions. Question 1 weighs 40 marks. Questions 2 and 3 weigh 30 marks each.

2. Only C++ is allowed.

3. Directory structure to be followed for submission:

   ```
   202620xxxx_A1
   |--202620xxxx_A1_Q1.cpp
   |--202620xxxx_A1_Q2.cpp
   |--202620xxxx_A1_Q3.cpp
   |--README.md
   ```

   Replace your roll number in place of `202620xxxx`.

4. Submission Format: Follow the above mentioned directory structure and zip the `RollNo_A1` folder and submit `RollNo_A1.zip` on Moodle.

   **Note:** All submissions which are not in the specified format will be awarded 0 in the assignment.

5. C++ STL (including vectors) is not allowed for any of the questions unless specified otherwise in the question. So `#include <bits/stdc++.h>` is not allowed.

6. A brief description of your approach for solving each question (data structures, algorithms, and any optimizations used) and step-by-step instructions on how to compile and execute your code for each question should be mentioned in README.

7. You can ask queries by posting on Moodle.

8. **Late Submission Rule:** Deadlines for assignments are final and will not be extended. Late submissions cost 5% loss of marks for each late day up to 3 days (i.e., 5%, 10%, ...). Submissions beyond 3 days from the deadline shall receive 0 marks.

**NOTE:** In case of plagiarism in any of the questions, the students involved will be awarded $-10$ as the final marks of Assignment 1.

# Question 1: Seam Carving [40 Marks]

## Problem Statement

Apply seam carving content-aware image-resizing algorithm on a given image. Take the height and width (in pixels) of the output image as inputs from the user.

## What is Seam Carving?

- Seam-carving is a content-aware image resizing technique where the image is reduced in size by one pixel of height (or width) at a time.
- A vertical seam in an image is a path of pixels connected from the top to the bottom with one pixel in each row.
- A horizontal seam is a path of pixels connected from the left to the right with one pixel in each column.

## Steps

- **Energy Calculation**: Each pixel has some RGB values. Calculate energy for each pixel. For example, you can use a dual-gradient energy function, but you are free to use any energy function of your choice. Refer to https://en.wikipedia.org/wiki/Seam_carving for details.
- **Seam Identification**: Identify the lowest energy seam.
- **Seam Removal**: Remove the lowest energy seam.

Figure 1: Original image before seam carving.

## Program Flow

1. Extract individual pixel's RGB values from the sample image. Load the RGB values in a 3D matrix (Height × Width × 3).
2. Apply seam carving algorithm.
3. Generate sample image output using the RGB values for resized image (New_Height × New_Width × 3).

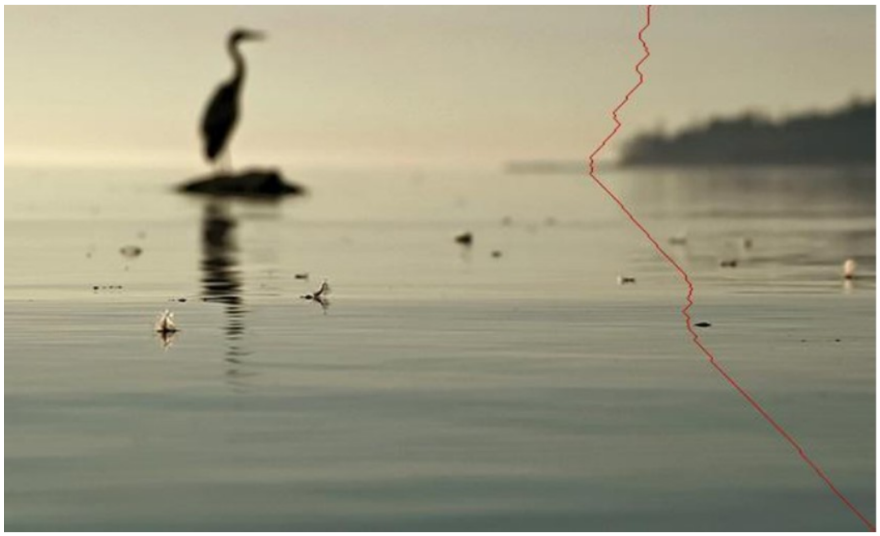

Figure 2: Image showing the lowest energy seam during preprocessing.

## Submission Format

- Only C++ is allowed.

## Evaluation Parameters

Accuracy and performance of the code.

## Note

1. Use OpenCV to extract RGB values of each pixel and to generate images back from RGB values.
2. To install the OpenCV C++ libraries, refer to https://docs.opencv.org/4.x/d7/d9f/tutorial_linux_install.html.
3. Implement your own seam carving algorithm.
4. C++ STL is not allowed (including vectors; design your own if required).
5. Images are provided in the `sample_input` folder, alongside the assignment PDF.
6. Reference Document: https://www.cs.princeton.edu/courses/archive/fall17/cos226/assignments/seam/index.html

Figure 3: Final resized image after seam carving.

## Question 2: Trie Harder [30 marks]

### Problem Statement

Design an efficient spell checker using the Trie data structure which supports the functionalities mentioned below.

1. **Spell Check**: Check if the input string is present in the dictionary.
2. **Autocomplete**: Find all the words in the dictionary which begin with the given input.
3. **Autocorrect**: Find all the words in the dictionary which are at an edit distance (Levenshtein distance) of at most 3 from the given input.

### Input Format

- First line will contain two space-separated integers $n$, $q$ which represent the number of words in the dictionary and the number of queries to be processed, respectively.
- Next $n$ lines will contain a single string $s$ which represents a word in the dictionary.
- Next $q$ lines will contain two space-separated values: an integer $a_i$ and a string $t_i$.
  - $a_i = 1$ means Spell Check operation needs to be done on $t_i$.
  - $a_i = 2$ means Autocomplete operation needs to be done on $t_i$.
  - $a_i = 3$ means Autocorrect operation needs to be done on $t_i$.
- Both strings $s$ and $t_i$ consist of lowercase English letters.

### Output Format

For each query, print the result in a new line.

- **Spell Check**: Print '1' if the string is present in the dictionary, otherwise '0'.
- **Autocomplete & Autocorrect**: Print the number of words in the first line. The following lines will be the set of words in lexicographical order.

### Constraints

- $1 \leq n \leq 1000$
- $1 \leq q \leq 1000$
- $1 \leq \text{len}(s) \leq 100$
- $1 \leq \text{len}(t_i) \leq 110$

### Sample Input

```
10 4
consider
filters
filers
entitled
tilers
litter
dames
filling
grasses
fitter
1 litter
1 dame
2 con
3 filter
```

### Sample Output

```
1
0
1
consider
5
filers
filters
fitter
litter
tilers
```

### Note

1. Only a trie should be used for storing the words in the dictionary.
2. You are allowed to use vector for this problem.

## Question 3: Battle of the Banners [30 Marks]

The annual Felicity Fest is in full swing along Research Street. Famous joints like David's Bakery, Vindhya Canteen, Domino's, Haldiram's, and Anna's Stall are locked in a fierce advertising war. Each stall has set up tall banners to grab attention, and these banners together form the silhouette of Research Street.

This silhouette is updated periodically on the giant LED screen inside the KRB building. But there's a twist, stalls keep increasing the heights of their banners throughout the day to stand out from their competition also new stalls getting added throughout the day.

Your task is to maintain the silhouette efficiently and answer queries about its shape. You will be given the initial positions and heights of all banners, followed by a sequence of updates and silhouette print requests.

### Input Format

- The first line contains an integer $q$
- The next $q$ lines each describe a query:
  - **Type 1 (Update):** `0 left_coordinate right_coordinate new_height` - Update the height of the banner for the given stall.
  - **Type 2 (Silhouette):** `1` - Print the current silhouette.

### Output Format

For each query of type 2, print the silhouette in the format:

$$\begin{matrix} x_1 & h_1 \\ x_2 & h_2 \\ \vdots & \vdots \\ x_k & h_k \end{matrix}$$

where $(x_j, h_j)$ marks a key point in the silhouette where the height changes.

### Constraints

- $1 \leq q \leq 10^5$
- $0 \leq l_i < r_i \leq 10^9$
- $0 \leq h_i \leq 10^9$
- It is guaranteed that coordinates are integers.
- You must implement an $O(\log N)$ update and $O(N)$ query mechanism, since printing will take $O(N)$.

## Example

**Input:**

```
8
0 0 5 10
0 2 7 15
0 6 9 18
1
0 2 7 20
1
0 5 15 25
1
```

**Output:**

```
0 10
2 15
6 18
9 0
0 10
2 20
7 18
9 0
0 10
2 20
5 25
15 0
```